GENETIC ALGORITHMS

Prof.dr. Henri Luchian

Recommended literature

The slides adapt presentations from:

Z. Michalewicz: *Genetic Algorithms + Data Structures = Evolution Programs*

M. Mitchell: *An Introduction to Genetic Algorithms*

D. Whitley: *A Genetic Algorithm Tutorial*

E. Goodman: *Introduction to Genetic Algorithms*

A.Eiben, J.Smith: *Introduction to Evolutionary Computing*

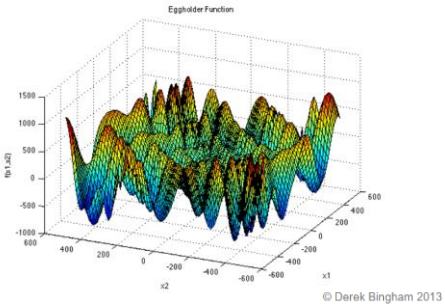
Strategies in problem-solving (1)

- Deterministic approach
  - **Exact determinist algorithms**: exact solutions
    - Complexity hierarchy for problems
      - Knapsack, TSP, MaxClique – each NP-complete
  - **Heuristics**: approximate solutions
    - Time complexity vs. precision for problems
      - Knapsack polynomial for any precision
      - TSP polynomial down to 50% precision
      - MaxClique **still exponential** for any precision!

Strategies in problem-solving (2)

- Probabilistic approach
  - **Probabilistic algorithms**
    - approximate solutions
    - Monte Carlo, Las Vegas, …
    - Evolutionary Algorithms, Hillclimbing, Simulated Annealing, Tabu Search…
    - Time complexity vs. precision in a statistical setting
      - Genetic Algorithms have a polynomial scheme;
      - some Evolutionary Strategies have probability 1 to find the exact solution

EGGHOLDER FUNCTION



$$f(\mathbf{x}) = -(x_2+47)\sin\left(\sqrt{\left|x_2+\frac{x_1}{2}+47\right|}\right) - x_1\sin\left(\sqrt{|x_1-(x_2+47)|}\right)$$

Eggholder function

Input Domain:

The function is usually evaluated on the square $x_i \in [-512, 512]$, for $i = 1, 2$.

Global Minimum:

$f(\mathbf{x}^*) = -959.6407$, at $\mathbf{x}^* = (512, 404.2319)$

Reference:

Global Optimization Test Functions Index. Retrieved June 2013, from http://infinity77.net/global_optimization/test_functions.html#test-functions-index.

Local search (maximization): Iterated Hillclimbing

begin t←0;
repeat
  local←FALSE;
  select a candidate solution (bitstring) $\mathbf{v}_c$ at random; evaluate $\mathbf{v}_c$;
  repeat generate the *length* strings Hamming-neighbors_1 of $\mathbf{v}_c$ {or select some among them};
    select $\mathbf{v}_n$, the one among these which gives the largest value for the objective function f;
    if f($\mathbf{v}_c$) < f($\mathbf{v}_n$) **then** $\mathbf{v}_c$←$\mathbf{v}_n$ **else** *local* ← TRUE
  until *local*;
  t←t+1;
until t=Max
end.

Mix of random search and deterministic search.
New idea: small mutation *on the representation*.

Local search (maximization): Simulated Annealing

begin
  t←0;
  initialize temperature T;
  select a current candidate solution (bitstring) $\mathbf{v}_c$ at random
  evaluate $\mathbf{v}_c$;
  repeat
    repeat select at random one of the *length* strings Hamming-neighbors_1 of $\mathbf{v}_c$;
      if f($\mathbf{v}_c$) < f($\mathbf{v}_n$) **then** $\mathbf{v}_c$←$\mathbf{v}_n$ **else**
      if *random* [0,1) < *exp* ((f($\mathbf{v}_n$) - f($\mathbf{v}_c$))/T) **then** $\mathbf{v}_c$←$\mathbf{v}_n$
    until *(termination condition);*
    T←g(T,t); {g(T,t)<T, ∀t}
    t←t+1
  until (*halting-criterion*)
end.

Little determinism remains. Diversity encouraged (=).
New ideas: the worse solution has a chance to survive (a smaller and smaller chance)

GA: Generalia

A non-classical approach

- Self-adaptation: optimisation as discovery (rather than invention)
- Less approximations
- More than one good solution
- Processing directly computer representations of solutions (not those of intermediary mathematical objects)
- Turning the combinatorial explosion to the user's benefit
- Meta-optimisation (meaning: optimisation of the problem-solving approach)

What are we dealing with ?

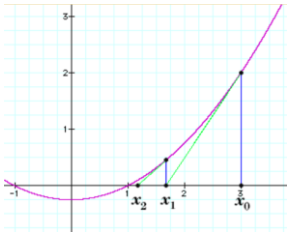
- "General-purpose" search methods
- Probabilistic algorithms
- Inspired from natural evolution
- Soft-computing paradigm
- Useful in
  - optimisation
  - machine learning
  - design of complex systems

Computing (1)

- Hard computing
  - Rigid, "static" models: no adaptation, or adaptation driven from outside
  - The model for a problem is completely described beforehand, not to be changed as solving goes on
    - Same "roadmap" for each instance of the problem
  - Calculus, Numerical Analysis (Newton's method), …

Hard Computing – an example: Newton's method

No adaptation – just mechanical repetition.
(nevertheless, a very good algorithm)



http://aleph0.clarku.edu/~djoyce/newton/newton.html

Computing (2)

- Soft computing
  - **Self-adapting** models
  - Only the frame for auto-adaptation is specified
  - Properties of the **instance** of the problem - fully exploited
  - Probabilistic modelling and reasoning
    - Evolutionary Computing
    - Artificial neural networks
    - Fuzzy sets

Less aproximation

Classical -*hard computing*

Understand the problem
Set up a model (*)
Devise theoretical results
Design an algorithm (*)
Code it, run **the program**
→ solution (*)

Evolutionary -*soft computing*

Understand the problem
Set up a model (*)
representation/function
Design the EA
Code, run **experiments**
→ solutions (o)

Optimisation

- Constraints
- Discrete or continuous variables
- Dynamic (moving) optimum
- Multicriterial
- Procedural view:
  - Step by step improvement
  - "Needle in the haystack"
- Criteria: accuracy and precision

Design of complex systems

- **Top-down** approach – the procedure:
  1. Design the general view of the system, with successive levels;
  2. While there are unvisited levels do
     - Descend one level and design that level.
- **Bottom-up** approach - alternatives:
  1. **Analytic / synthetic** procedure:
     - design one component at a time
     - join the components together (level by level)
  2. **"Evolutionary"** procedure:
     - Design a frame for evolution
     - Design very basic features / components
     - Let the components evoluate (the system evolves itself)
- Examples: ANN, sorting networks etc.

Natural evolution: **genetics**

- 1859: Darwin– natural selection
  - "the Jenkins nightmare": if combination is like mixing liquid paints, then … homogenous population (no selection / improvement possible).
- 1865: Mendel : Genetics - **discrete**, not continuous
- 1900: de Vries, Correns, von Tschermak – rediscover Mendel's Genetics. Mutation Theory
- 1915: T. Morgan - *The Mechanism of Mendelian Heredity* - joins heredity and genetics: chromosomes as sequences of genes
  - "Morgan's theory of the chromosome represents a great leap comparable with Galileo or Newton" – C.H.Waddington.
- 1920's: Fisher, Wright, Haldane – join *natural selection* and *genetics*: **evolution of populations**.

Rechenberg's Conjecture

- "Natural Evolution is – or contains – an optimisation process which is so powerful that it is worth simulating it for solving very hard optimisation problems."

General Scheme of a GA

```
begin
    t←0;
    Initialise P(t);
    Evaluate P(t);
    While not (halting condition) do begin
        t←t+1;
        Select P(t) from P(t-1);
        Apply operators to P(t);
        Evaluate P(t)
    end
end.
```

- **Except possibly for the Evaluation step, the scheme is polynomial.**
- **This scheme is valid for (most) evolutionary algorithms.**

Glossary (1)

- Chromosome (individual) – a sequence of genes (…)
- Gene – atomic information in a chromosome
- Locus – position of a gene
- Allele – all posible values for a locus
- Mutation – elementary random change of a gene
- Crossover (recombination) – exchange of genetic information between *parents*
- Fitness function – actual function to optimise; *environment*

Glossary (2)

- Population – set of candidate solutions
- Generation – one iteration of the evolutionary process
- Parent – chromosome which "creates" offspring via a genetic operator
- Descendant (offspring) – chromosome obtained after applying a genetic operator to one or more parents
- Selection – (random) process of choosing parents and / or survivors
- Genotype – representation at the individual level
- Phenotype – (theoretical) object represented through genotype
- Solution given by a GA – best candidate in the last generation

Evolutionary computing

- Evolution Strategies
- Evolutionary Programming
- Genetic Algorithms
  - Genetic Programming
- Differential Evolution

The first two major directions

- Evolution strategies
  - A method to optimise real-valued parameters
  - Random mutation and selection of the fittest
  - Ingo Rechenberg (1965,1973); Schwefel (1975,1977)
- Evolutionary programming
  - Fogel, Owens, Walsh (1966)
  - Random mutations on the state-transition diagrams of finite state machines; selection of the fittest

A Breakthrough: GAs

- John Holland – 60's and 70's.
  - In search of evolvable hardware
- Goal: not solving specific problems, but rather study adaptation
- "Adaptation in Natural and Artificial Systems" (1975)
- Abstraction of biological evolution
- Theoretical study (schema theorem).

Further ideas

- Messy Genetic Algorithms (Goldberg)
- Genetic Programming (Koza)
- Co-evolution (Hillis)
- Memetic algorithms

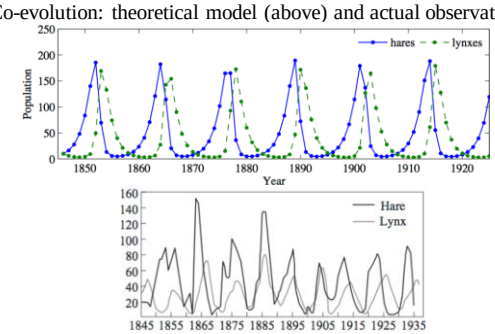
- **Metaphor-based algorithms**

- Statistical study of "suitability" between representation and fitness function, operators, etc.

Co-evolution

applied to Sorting Networks

Co-evolution: theoretical model (above) and actual observations



If – at least occasionally - improvements in one species become an evolutionary factor for the other species, then the cycles are super-seeded by an overall improvement of each species: **population optimisation**.

Co-evolution in Genetic Algorithms

Introduced in 1990 by Danny Hillis:

"*Co-evolving Parasites Improve Simulated Evolution as an Optimization Procedure*"

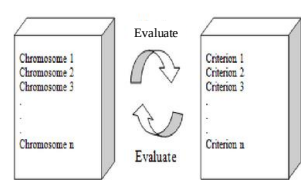
While pursuing the very difficult problem of **optimising Sorting Networks** (see next slides):

- stage 1. a Genetic Algorithm where the population consists of (variable length) representations of sorting networks. Weak results.

- stage 2. Co-evolution. Two Genetic Algorithms running in parallel: the first GA had a population of (better-and-better) candidate sorting networks; the second GA had a population of (harder and harder) test cases for sorting.

The goal: the solutions offered by the two Genetic Algorithms should be a) (near-)optimal sorting networks and b) most difficult instances of the sorting problem.

Co-evolution in GA



GA-1: Target population

GA-2: Co-evolved (test) population

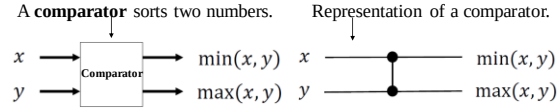
https://www.semanticscholar.org/paper/Coevolution-Evolutionary-Algorithm%3A-A-Survey-Kavetha/

Sorting Networks

- (Correct) sorting algorithms which:
  1. only compare and switch and
  2. do this only on predefined positions.
- Requirements: minimal **size** (number of comparisons) / **depth** (number of parallel steps)
- Then: assymptotically O(*n*log*n*)
  - Which is useless for optimising SN of a given input length.
    - Sorting Networks are length-specialised algorithms

Comparator; Sorting Network

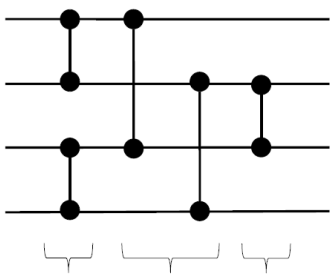
A **comparator** sorts two numbers. Representation of a comparator.



©Zwick Tel Aviv University

An **n-input sorting network** is a set of inter-waved comparators which collectively sort correctly any ordered input of **n** numbers.

- There are **n** input numbers, each feeding a single comparator.
- Each output of a comparator is either an global output wire, or feeds a single comparator.
- The sorting network must be *acyclic*. The number of output wires is **n**.



A **4-input sorting network** of
size 5 (has five comparators) and **depth 3** (works in three parallel steps).

Problem (Hillis)**: design a size-optimal 16-input sorting network**, using Genetic Algorithms.

©Zwick TAU

Size- and Depth- Optimal SNs

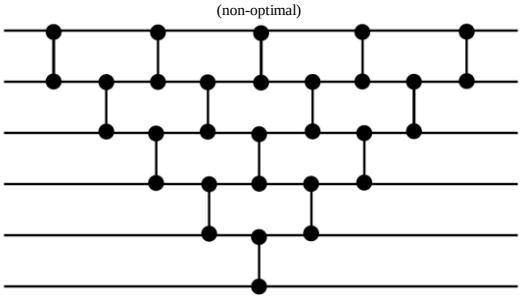
input lengths from 1 to 17

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Depth	0	1	3	3	5	5	6	6	7	7	8	8	9	9	9	9	10
Size, upper bound	0	1	3	5	9	12	16	19	25	29	35	39	45	51	56	60	71
Size, lower bound (if different)											33	37	41	45	49	53	58

For input length above 10, size-optimal SNs are not yet known.

The 16-input Bubble-Sorting Network

(non-optimal)



This sorting network has **size** n*(n-1)/2 and **depth** 2*n-1.
For n=16, one gets: size=136, depth=33. **Optimal: size<=60, depth=9**.

©Zwick TAU University

- The first sixteen **depth-optimal** networks are listed in Knuth's *Art of Computing Programming* since the 1973 edition;
- the optimality of the first eight was established at the time, by Floyd and Knuth in the 1960s
- in 1973, this property wasn't proven for the final six; it took until 2014 to do that.

GA and Sorting Networks

- D. Knuth mentions that **depth-optimal** sorting networks

  for $n = 9$ and $n = 11$

  were found by **Loren Schwiebert** in **2001**

  using **Genetic Algorithms**.

Co-evolution for Sorting Networks

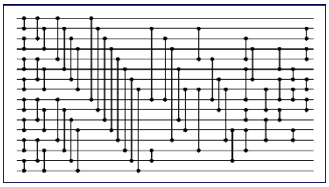
- GA1: Population1 – candidate sorting networks.
- GA2: Population2 – candidate test cases.
- Fitness1: (max) nr.-solved-test-cases from Population2.
  - for each candidate sorting network from Population1
- Fitness2: (min) nr.-full-solvers from Population1
  - For each candidate test case from Population2.

Co-evolution for Sorting Networks

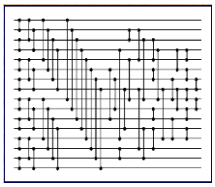
- Representation for a bubble-sort 16-sorting network:
- (1,2)(1,3)(1,4)(1,5)…(1,16)(2,3)…(15,16)

- Representation for a test-case:
- (16, 15, 14, …, 1).

Sorting Networks



Hillis' construction (**co-evolution**): 61-comparator ,11 parallel steps 16-input sorting network.



Green's contruction: 60-comparator, 10 parallel steps 16-input s.n.

Evolutionary Computing - Shared Features

- maintain a <u>population</u> of representations of <u>candidate solutions</u>
- which are improved over successive <u>generations</u>
- by means of <u>mutation (, crossover,…)</u>
- and <u>selection</u> based on individual merit
- assigned to individuals through <u>evaluation</u> against <u>fitness function</u> (“environment”)

Data Structures

- Genetic Algorithms
  - standard: bitstring representations (integers, real numbers, permutations etc.);
  - others: bidimensional string structures (graphs, matrices etc.); neighborhood-based (“island”); “geographically”-distributed etc.
  - GP: trees (programs)
- Evolution Strategies: floating point representations (real numbers)
- Evolution Algorithms: finite state machines

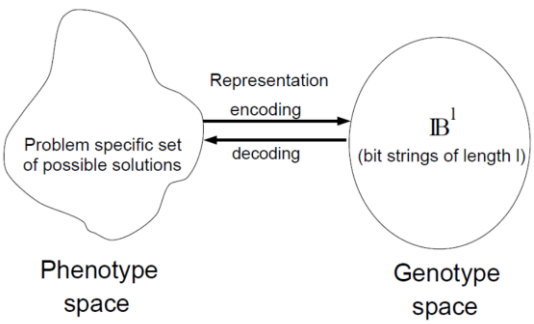
Theoretical Models

- Schema model (Holland)
  - Scheme;
  - Building-blocks hypothesis
  - Implicit parallelism
- Markov-chain models (→linked processes)
- Vose models (statistical mechanics)
- Bayesian models (Muehlenbein, …)

The Elements of a Genetic Algorithm

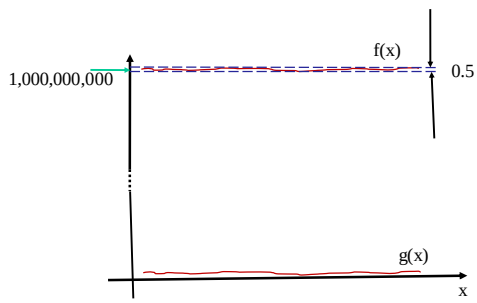
- A population of *pop_size* individuals (chromosomes)
  - An initialisation procedure
- A representation (genotype) of candidate solutions (phenotype)
- An evaluation (fitness) function
- A selection scheme
- Genetic operators (mutation, crossover, …)
- Parameters (size, rates, scalability, …)
- A halting condition

Representation



©Thomas Back, Leiden University

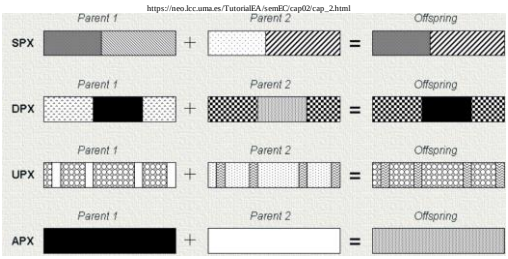
Fitness function



Fitness: g(x) = f(x) – 999,999,999

Some crossover operators

https://neo.lcc.uma.es/TutorialEA/semEC/cap02/cap_2.html



Single-Point, Double-Point, Uniform and Arithmetic Crossover Operators.
Only one offspring is shown (SPX, DPX, UPX).

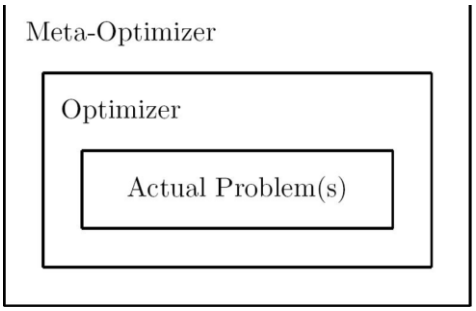
Parameters

Population size	100
Number of generations	100
Crossover rate	0.76
Mutation rate	0.001
Type of crossover	Single point crossover
Chromosome length	24
Selection strategy	Roulette wheel selection

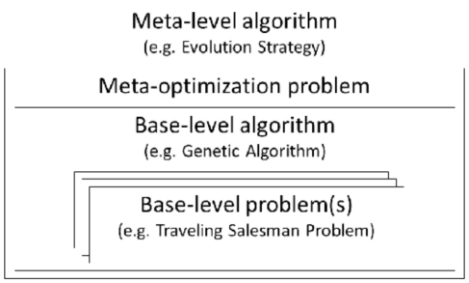
- How do we know the optimal value for each parameter?
- How do we know which of the other options for various elements of the GA, are optimal?

Meta-optimisation

- Design of a GA for a given problem:
  - choose a representation (out of a few possible);
  - choose a fitness function (various models; scalable);
  - choose a selection scheme (around 15 "classical");
  - choose operators (or invent them);
  - set the parameter values (tens/hundreds).
- A moderate evaluation: 5*10*15*10*100≈750.000: hundreds of thousands of GAs for a given problem.
- Most of them probably perform poorly, but usually a few of them outperform other kinds of algorithms
- Finding the "good" GA for a problem is an optimisation problem in itself
- How can one find it?

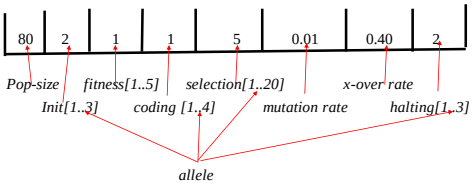


Pedersen, M.E.H., Tuning & Simplifying Heuristical Optimization, PhD Thesis, 2010, University of Southampton



Christoph Neumüller, S. Wagner, G. Kronberger, M. Affenzeller: *Parameter Meta-optimization of Metaheuristic Optimization Algorithms*, published in EUROCAST 2011

Meta-chromosome



Meta-GA

- Meta-population
- Meta-chromosomes
- Meta-mutation
- Meta-crossover
- **Meta-evaluation**
- Meta-selection

- Few generations (because evaluation is time consuming)
- NOT meta-meta… (would bring very little improvement)

- Meta_GA scheme (running for **n** generations):
  - Initialise meta-population Meta_Pop(**0**)
  - **Meta_evaluate** each meta-chromosome in Meta_Pop(**0**)
  - for **g**=**1**..**n** do
    - Meta-select Meta_Pop(**g**) from Meta-Pop(**g-1**)
    - Apply Meta-operators on meta-chromosomes in Meta_Pop(**g**)
    - Meta-evaluate each meta-chromosome in Meta_Pop(**g**)
- **Meta_evaluate** procedure (at meta-generation **g**):
  - for **i**=**1**..**meta_pop-size** do
    - Set up the Basic_GA(**i**,**g**) according to the meta-genes of meta-chromosome(**i**,**g**)
    - Run **k** times Basic_GA(**i**,**g**) and get solutions sol(**i**,**g**,**1**), sol(**i**,**g**,**2**)...sol(**i**,**g**,**k**)
    - meta_eval(chromosome(**i**,**g**)) is the *average* or the *maximum* of these **k** solutions